RiciclApp

Deliverable D2

Gruppo n°: 20

Componenti:

Studente: Binco Francesco (Matricola: 243781)
Studente: Cortese Riccardo (Matricola: 244656)
Studente: Giovagnetti Lorenzo (Matricola: 247579)

Progetto di Ingegneria del Software - Prof. Sandro Luigi Fiore

Anno Accademico 2025/2026

Sommario

Il presente progetto descrive l'ideazione e la progettazione di un sistema per l'ottimizzazione della raccolta differenziata urbana.

L'obiettivo principale dell'applicazione è duplice: da un lato, semplificare il conferimento dei rifiuti per il cittadino attraverso strumenti di riconoscimento (scansione barcode) e localizzazione dei punti di raccolta con relativo stato di riempimento; dall'altro, fornire agli enti comunali e alle aziende di raccolta una visione d'insieme in tempo reale dello stato del territorio.

Grazie ai dati trasmessi dai sensori IoT installati nei bidoni, l'amministratore può monitorare i livelli di riempimento, identificare le zone critiche e pianificare interventi di svuotamento mirati. Questo approccio permette di passare da una raccolta a calendario a una gestione dinamica "on-demand", riducendo i costi operativi e l'impatto ambientale dei mezzi di trasporto.

Il sistema si avvale di una rete di sensori IoT installati nei bidoni stradali per il monitoraggio in tempo reale dei livelli di riempimento, permettendo agli operatori ecologici di usufruire di percorsi di raccolta ottimizzati.

Indice

1 Diagramma componenti **1**
1.1 Diagramma dei componenti . . . 1
1.2 Descrizione dei componenti . . . 1
1.2.1 Gestione utente . . . 1
1.2.2 Gestione bidoni . . . 3
1.2.3 Gestione segnalazioni . . . 3
1.2.4 Gestione evento raccolta . . . 4

2 Diagramma delle classi e descrizione delle classi **6**
2.1 Diagramma delle classi . . . 6
2.2 Gruppo classi autenticazione e profilo . . . 6
2.2.1 Classe Utente . . . 6
2.2.2 Classe UtenteRegistrato . . . 7
2.2.3 Classe OperatoreEcologico . . . 7
2.2.4 Classe Admin . . . 8
2.2.5 Classe Autenticazione . . . 8
2.3 Gruppo classi centri raccolta e sensori IoT . . . 9
2.3.1 Classe Bidone . . . 9
2.3.2 Classe Sensore . . . 9
2.4 Gruppo classi segnalazione . . . 10
2.4.1 Classe Segnalazione . . . 10
2.5 Gruppo classi eventi . . . 10
2.5.1 Classe EventoRaccolta . . . 10
2.6 Gruppo classi premi . . . 11
2.6.1 Classe Premio . . . 11
2.6.2 Classe Voucher . . . 11
2.7 Gruppo classi prodotto . . . 12
2.7.1 Classe Prodotto . . . 12

3 Constraint OCL **13**
3.1 Classe Autenticazione . . . 13
3.1.1 Login . . . 13
3.1.2 Logout . . . 13
3.1.3 Controllo Username e Password . . . 13
3.2 Classe Utente . . . 13
3.2.1 ID con valore positivo . . . 13
3.2.2 Password sicura . . . 13
3.3 Classe Utente Registrato . . . 13
3.3.1 Punti sempre positivi . . . 13
3.3.2 Riscattare premi . . . 14
3.4 Classe Operatore Ecologico . . . 14
3.4.1 Controllo Matricola . . . 14
3.5 Classe Bidone . . . 14
3.5.1 Livello di Riempimento . . . 14
3.6 Classe Sensore . . . 14
3.6.1 Livello Batteria . . . 14

3.6.2 Capacità Residua 14
3.6.3 Ultimo Aggiornamento 14
3.7 Classe Segnalazione 15
3.7.1 Descrizione non vuota 15
3.7.2 Data Corretta 15
3.8 Classe Premio 15
3.8.1 Costo sempre positivo 15
3.9 Classe Voucher 15
3.9.1 Data di scadenza sempre futura 15
3.9.2 Il codice non deve essere vuoto 15

1 Diagramma componenti

1.1 Diagramma dei componenti

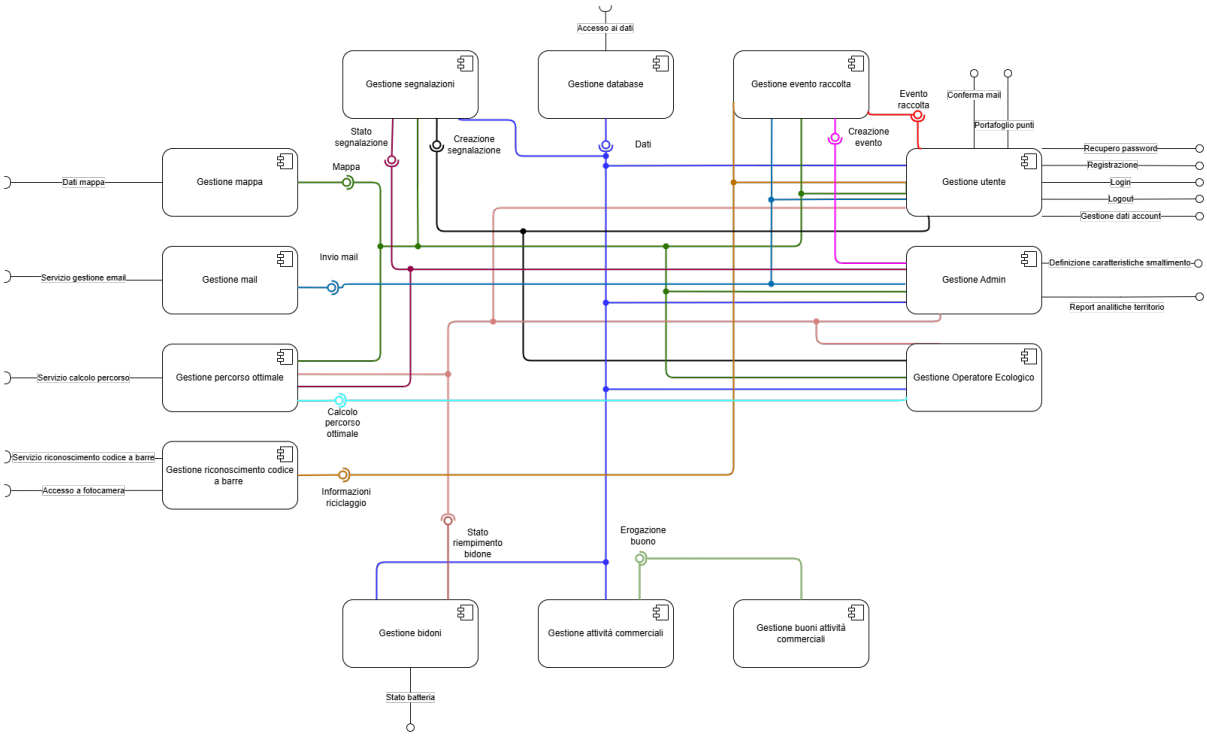

Figura 1: Diagramma dei componenti

1.2 Descrizione dei componenti

1.2.1 Gestione utente

Questo componente permette di gestire tutte le funzionalità richieste da un utente generico.

Tabella 1: Tabelle descrizione componente per gestione utente

Tipologia	Nome	Descrizione
`Fornita`	`Login`	Il componente permette ad un utente che in precedenza abbia effettuato una registrazione, quindi in possesso di un account, di effettuare l'accesso tramite username e password

Tipologia	Nome	Descrizione
`Fornita`	`Logout`	Il componente permette ad un utente che in precedenza abbia effettuato il login, quindi in possesso di un account, di effettuare la disconnessione al proprio account per la sessione in corso
`Fornita`	`Registrazione`	Il componente deve permettere ad un utente senza un account associato alla propria mail di creare un account (*RF5*)
`Fornita`	`Recupero password`	Il componente deve permettere ad un utente di recuperare la password in caso l'avesse smarrita
`Fornita`	`Conferma mail`	Il componente deve permettere ad un utente di confermare la propria mail nel momento in cui esso crea un nuovo account (Registrazione)
`Fornita`	`Portafoglio punti`	Il componente tiene traccia di tutti i punti che un utente in possesso di account accumula. Tali punti vengono accreditati in caso di segnalazioni corrette, partecipazioni ad eventi raccolta e per smaltimenti di prodotti (*RF7*). L'utente inoltre può convertire tali punti in buoni sconto offerti dalle attività commerciali (*RF9*)
`Fornita`	`Gestione dati account`	Il componente deve permettere all'utente in possesso di account di modificare i dati del proprio account, come ad esempio foto profilo e dati anagrafici (*RF5*)
`Richiesta`	`Dati`	Il componente necessita di dati presenti nel database per la gestione dell'utente registrato
`Richiesta`	`Mappa`	Il componente necessita di un servizio che permetta di visualizzare la mappa. In questo modo l'utente può visualizzare i centri di raccolta del territorio (*RF1*)
`Richiesta`	`Invio notifica`	Il componente necessita di un servizio che permetta di ricevere notifiche nel momento in cui vi siano errori, come ad esempio errori di inserimento di dati
`Richiesta`	`Informazioni riciclaggio`	Il componente necessita di un servizio che dia informazioni sul corretto smaltimento di rifiuti tramite il riconoscimento del codice a barre. Il riconoscimento può avvenire tramite scansione dell'immagine oppure tramite inserimento testuale (*RF3, RF4*)

Tipologia	Nome	Descrizione
`Richiesta`	`Stato riempimento bidone`	Il componente necessita di un servizio che permetta all'utente di visualizzare lo stato di riempimento dei bidoni nei vari centri (*RF1, RF2*)
`Richiesta`	`Creazione segnalazione`	Il componente necessita di un servizio che permetta all'utente di effettuare segnalazioni come ad esempio problemi con bidoni (*RF6*)
`Richiesta`	`Evento raccolta`	Il componente necessita di un servizio che permetta all'utente la partecipazione di un evento di raccolta *[nuova aggiunta rispetto a D1]*
`Richiesta`	`Invio mail`	Il componente necessita di un servizio che inivii mail in modo che l'utente possa ricevere delle mail come ad esempio per la conferma della mail

1.2.2 Gestione bidoni

Questo componente permette di gestire le funzionalità per la gestione dei sensori che controllano lo stato di riempimento del bidone.

Tabella 2: Tabelle descrizione componente per gestione bidoni

Tipologia	Nome	Descrizione
`Fornita`	`Stato batteria`	Il componente permette di visualizzare lo stato della batteria del sistema di controllo del bidone (*RF19*)
`Fornita`	`Stato riempimento bidone`	Il componente permette di visualizzare lo stato di riempimento del bidone (*RF19*)
`Richiesta`	`Dati`	Il componente necessita di dati presenti nel database per la gestione dei vari bidoni disposti nel territorio, come ad esempio posizione o tipologia

1.2.3 Gestione segnalazioni

Questo componente permette di gestire le funzionalità per la gestione delle segnalazioni.

Tabella 3: Tabelle descrizione componente per gestione delle segnalazioni

Tipologia	Nome	Descrizione
`Fornita`	`Stato segnalazione`	Il componente permette di visualizzare lo stato della segnalazione, ad esempio "Presa in carico", "In corso", "Risolta"
`Fornita`	`Creazione segnalazione`	Il componente permette di creare una nuova segnalazione. Essa deve essere accompagna da una descrizione, da una foto (opzionalmente) e deve essere geolocalizzata
`Richiesta`	`Dati`	Il componente necessita di dati presenti nel database per la gestione delle varie informazioni come, ad esempio, la localizzazione dei vari bidoni
`Richiesta`	`Mappa`	Il componente necessita di un servizio che permetta di visualizzare la mappa. In questo modo si potrà effettuare una segnalazione per uno specifico bidone localizzato tramite la mappa

1.2.4 Gestione evento raccolta

Questo componente permette di gestire le funzionalità per la gestione di eventi di raccolta.

Questo componente è aggiuntivo rispetto al precedente documento (*D1*) a seguito della creazione di un'ulteriore funzionalità.

La funzionalità "*Evento raccolta*" permette all'admin di creare un evento di raccolta di rifiuti abbandonati, circoscritti in un'area prescelta. Ogni utente può decidere se partecipare o meno e, a seconda della quantità di rifiuti raccolti nell'area, accumulerà punti aggiuntivi.

Tabella 4: Tabelle descrizione componente per evento raccolta

Tipologia	Nome	Descrizione
`Fornita`	`Creazione evento`	Il componente permette ad un utente admin di creare l'evento di raccolta. L'admin definisce la zona in cui l'evento avviene
`Fornita`	`Evento raccolta`	Il componente permette ad un utente in possesso di account di partecipare ad un evento di raccolta creato
`Richiesta`	`Mappa`	Il componente necessita di un servizio che permetta di visualizzare la mappa. In questo modo l'utente potrà visualizzare l'area in cui l'evento è in corso e i bidoni presenti

Tipologia	Nome	Descrizione
`Richiesta`	`Invio mail`	Il componente necessita di un servizio che inivii mail in modo che gli utenti partecipanti all'evento siano notificati
`Richiesta`	`Informazioni riciclaggio`	Il componente necessita di un servizio che dia informazioni sul corretto smaltimento di rifiuti tramite il riconoscimento del codice a barre. Il riconoscimento può avvenire tramite scansione dell'immagine oppure tramite inserimento testuale (*RF3, RF4*)

2 Diagramma delle classi e descrizione delle classi

In questa sezione viene riportata un'analisi delle classi richieste per implementare il sistema mediante l'utilizzo del diagramma delle classi. Per una maggiore leggibilità viene prima riportato un diagramma complessivo e poi un'analisi su alcuni gruppi di classi. Per ogni classe/le classi principali vengono specificati attributi e operazioni con relative descrizioni (parametri e valori di ritorno vengono omessi per questione di maggiore chiarezza della leggibilità).

2.1 Diagramma delle classi

La Figura 2 mostra il diagramma delle classi del sistema RiciclApp.

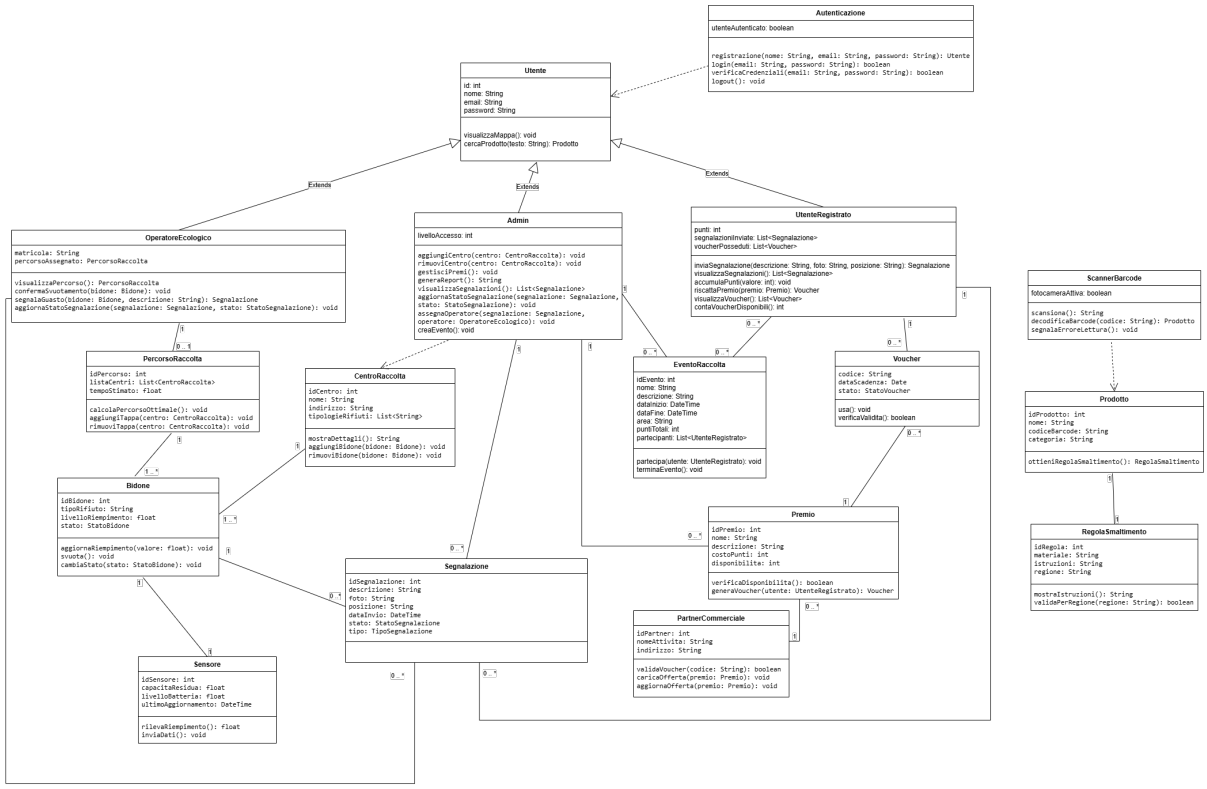

Figura 2: Diagramma delle classi di RiciclApp

2.2 Gruppo classi autenticazione e profilo

Vengono analizzate qui le classi che si occupano della gestione dell'autenticazione e del profilo utente.

2.2.1 Classe Utente

La classe `Utente` rappresenta un utente generico che sfrutta le funzionalità dell'app. Detiene i dati di base necessari a differenziare i vari utenti, fornendo funzionalità di base come visualizzazione della mappa e ricerca delle regole di smaltimento del prodotto.

Attributi

- `id: int` – numero identificativo univoco per ogni utente;
- `nome: String` – nome dell'utente;
- `email: String` – email personale;
- `password: String` – password personale.

Operazione	Descrizione
`visualizzaMappa()`	Consente all'utente di visualizzare la mappa che mostra i centri di raccolta vicini.
`cercaProdotto()`	Consente all'utente di ricercare le regole di smistamento di un prodotto.

2.2.2 Classe UtenteRegistrato

La classe `UtenteRegistrato` rappresenta un utente che ricopre nel sistema il ruolo di cittadino che svolge la raccolta differenziata. Estende la classe `Utente` e introduce funzionalità aggiuntive come l'invio di segnalazioni, l'accumulo di punti e il riscatto di premi.

Attributi

- `punti: int` – numero totale di punti accumulati dall'utente;
- `segnalazioniInviate: List<Segnalazione>` – elenco delle segnalazioni effettuate;
- `voucherPosseduti: List<Voucher>` – insieme dei voucher riscattati.

Operazione	Descrizione
`inviaSegnalazione()`	Consente all'utente di creare ed inviare una nuova segnalazione geolocalizzata.
`visualizzaSegnalazioni()`	Restituisce l'elenco delle segnalazioni effettuate.
`accumulaPunti()`	Aggiorna il punteggio dell'utente in base alle azioni svolte.
`riscattaPremio()`	Permette di convertire i punti accumulati in un voucher.
`visualizzaVoucher()`	Elenca la lista di voucher posseduti.
`contaVoucherDisponibili()`	Permette di calcolare il numero di voucher disponibili.

2.2.3 Classe OperatoreEcologico

La classe `OperatoreEcologico` rappresenta un utente che ricopre il ruolo di operatore ecologico, che svolge la raccolta dei rifiuti, nel sistema . Estende la classe `Utente` e introduce funzionalità aggiuntive come la visualizzazione di un percorso di raccolta personalizzato, la conferma di avvenuto svuotamento dei bidoni e l'invio di segnalazioni riguardo il loro stato.

Attributi

- `matricola: String` – numero di matricola dell'operatore;
- `percorsoAssegnato: PercorsoRaccolta` – percorso personalizzato da seguire.

Operazione	Descrizione
`visualizzaPercorso()`	Mostra il percorso personalizzato da seguire.
`confermaSvuotamento()`	Consente all'operatore di segnalare l'avvenuto svuotamento.
`segnalaGuasto()`	Permette di creare ed inviare una segnalazione di un guasto del bidone.
`aggiornaStatoSegnalazione()`	Consente di confermare l'aggiornamento dello stato di un bidone a seguito di una segnalazione.

2.2.4 Classe Admin

La classe `Admin` rappresenta un utente che ricopre il ruolo di amministratore, che gestisce le segnalazioni, i centri e i premi, nel sistema. Estende la classe `Utente` e introduce funzionalità aggiuntive come l'aggiunta di centri di raccolta e la gestione delle segnalazioni.

Attributi

- `livelloAccesso: int` – livello al quale opera.

Operazione	Descrizione
`aggiungiCentro()`	Aggiunge un nuovo centro di raccolta sulla mappa.
`rimuoviCentro()`	Rimuove un centro di raccolta dalla mappa.
`gestisciPremi()`	Permette la gestione dell'assegnazione di premi.
`generaReport()`	Consente di generare report sullo stato dei bidoni e delle segnalazioni.
`visualizzaSegnalazioni()`	Permette di visualizzare le varie segnalazioni.
`aggiornaStatoSegnalazione()`	Consente di aggiornare lo stato di un bidone a seguito di una segnalazione.
`assegnaOperatore()`	Permette di assegnare un operatore per svolgere una certa mansione.
`creaEvento()`	Permette di creare un evento a tempo limitato.

2.2.5 Classe Autenticazione

La classe `Autenticazione` verifica e conferma l'autenticazione di un utente nel sistema. Dipende dalla classe `Utente` e introduce le funzionalità di registrazione, login, verifica e logout.

Attributi

- `utenteAutenticato: boolean` – indica se un utente è autenticato o meno.

Operazione	Descrizione
`registrazione()`	Permette di registrarsi nel sistema.
`login()`	Consente all'utente di connettersi all'applicazione.
`logout()`	Consente all'utente di disconnettersi dall'applicazione.
`verificaCredenziali()`	Verifica le credenziali inserite dall'utente.

2.3 Gruppo classi centri raccolta e sensori IoT

Vengono analizzate qui le classi che si occupano della gestione dei bidoni e dei sensori IoT.

2.3.1 Classe Bidone

La classe `Bidone` rappresenta un contenitore per la raccolta differenziata e le sue caratteristiche. Le sue funzionalità servono ad aggiornare lo stato di riempimento del bidone.

Attributi

- `idBidone: int` – identificativo numerico univoco.
- `tipoRifiuto: String` – per specificare quale tipo di rifiuto viene raccolto all'interno del bidone.
- `livelloRiempimento: float` – indica il livello di riempimento del bidone.
- `stato: StatoBidone` – indica lo stato di pienezza del bidone.

Operazione	Descrizione
`aggiornaRiempimento()`	Aggiorna il livello di riempimento del bidone.
`cambiaStato()`	Modifica lo stato del bidone.
`svuota()`	Rappresenta l'operazione di svuotamento del bidone.

2.3.2 Classe Sensore

La classe `Sensore` rappresenta un sensore e le sue funzionalità, che servono a determinare lo stato di riempimento del bidone e inviare tali dati al sistema per contribuire allo sviluppo di un percorso di raccolta dinamico.

Attributi

- `idSensore: int` – identificativo numerico univoco.
- `capacitaResidua: float` – per specificare la capacità del bidone rimanente.
- `livelloBatteria: float` – indica il livello di carica della batteria.
- `ultimoAggiornamento: DateTime` – indica quando è avvenuto l'ultimo aggiornamento.

Operazione	Descrizione
`rilevaRiempimento()`	Rileva il livello di riempimento del bidone.
`inviaDati()`	Invia al sistema i dati sullo stato di pienezza del bidone.

2.4 Gruppo classi segnalazione

Viene analizzate la classe riguardante le segnalazioni.

2.4.1 Classe Segnalazione

La classe `Segnalazione` descrive le segnalazioni che vengono inviate dagli utenti registrati e dagli operatori ecologici al sistema.

Attributi

- `idSegnalazione: int` – identificativo numerico univoco.
- `descrizione: String` – descrizione dell'evento segnalato.
- `foto: String` – riferimento all'eventuale immagine associata alla segnalazione.
- `posizione: String` – luogo in cui si è verificato l'evento.
- `dataInvio: DateTime` – data e ora di invio della segnalazione.
- `stato: StatoSegnalazione` – stato di elaborazione della segnalazione.
- `tipo: TipoSegnalazione` – tipologia della segnalazione.

Operazione	Descrizione
`invia()`	Invia al sistema la segnalazione compilata.

2.5 Gruppo classi eventi

Vengono qui analizzate le classi relative ai premi.

2.5.1 Classe EventoRaccolta

La classe `EventoRaccolta` rappresenta un evento inizializzato dall'Admin al quale diversi utenti possono partecipare per ottenere punti extra svolgendo azioni.

Attributi

- `idEvento: int` – identificativo numerico univoco.
- `nome: String` – nome dell'evento.
- `descrizione: String` – descrizione dell'evento.
- `dataInizio: DateTime` – data e ora di inizio dell'evento.

- `dataFine: DateTime` – data e ora di fine dell'evento.
- `area: String` – zona dove avviene l'evento.
- `puntiTotali: int` – punti totali racimolati alla fine dell'evento.
- `partecipanti: List<UtenteRegistrato>` – lista dei partecipanti all'evento.

Operazione	Descrizione
`partecipa()`	Registra la partecipazione di un utente registrato all'evento.
`terminaEvento()`	Segna la conclusione dell'evento.

2.6 Gruppo classi premi

Vengono qui analizzate le classi relative ai premi.

2.6.1 Classe Premio

La classe `Premio` rappresenta un premio ottenibile dagli utenti tramite i punti accumulati e consente la generazione dei relativi voucher.

Attributi

- `idPremio: int` – identificativo numerico univoco.
- `nome: String` – nome del premio.
- `descrizione: String` – descrizione del premio.
- `costoPunti: int` – punti necessari per ottenere il premio.
- `disponibilita: int` – numero di premi disponibili.

Operazione	Descrizione
`verificaDisponibilita()`	Verifica se il premio è disponibile per il riscatto.
`generaVoucher()`	Genera un voucher associato al premio selezionato.

2.6.2 Classe Voucher

La classe `Voucher` rappresenta un buono utilizzabile presso un partner commerciale che lo accetta.

Attributi

- `codice: String` – codice del voucher.
- `dataScadenza: Date` – data di scadenza del voucher.
- `stato: StatoVoucher` – stato di utilizzo del voucher.

Operazione	Descrizione
`usa()`	Permette di usare il voucher, aggiornando il suo stato.
`verificaValidita()`	Verifica la validità del voucher.

2.7 Gruppo classi prodotto

Vengono qui analizzate le classi relative ai prodotti (viene solo riportata la classe prodotto per semplicità).

2.7.1 Classe Prodotto

La classe `Prodotto` rappresenta un prodotto presente nel sistema, identificato tramite codice a barre e associato a specifiche regole di smaltimento.

Attributi

- `idProdotto: int` – identificativo numerico univoco.
- `nome: String` – nome del prodotto.
- `codiceBarcode: String` – codice a barre del prodotto.
- `categoria: String` – categoria del prodotto.

Operazione	Descrizione
`ottieniRegolaSmaltimento()`	Restituisce le regole di smaltimento associate al prodotto scansionato.

3 Constraint OCL

In questa sezione vengono analizzati in linguaggio OCL i vincoli legati al diagramma delle classi. Questi vincoli rappresentano invarianti, precondizioni e postcondizioni sulle operazioni delle classi.

3.1 Classe Autenticazione

3.1.1 Login

Descrizione: l'operazione di Login(String username, String Password) setta l'attributo autenticato a true.

```
context Autenticazione::login(string username, string password): boolean
post: utenteAutenticato = true
```

3.1.2 Logout

Descrizione: l'operazione di Logout setta l'attributo autenticato a false.

```
context Autenticazione::logout()
post: utenteAutenticato = false
```

3.1.3 Controllo Username e Password

Descrizione: lo Username e la Password non devono mai essere vuoti.

```
context Autenticazione::login(string username, string password): boolean
pre: username <> "" and password <> ""
```

3.2 Classe Utente

3.2.1 ID con valore positivo

Descrizione: L'ID dell'utente deve essere sempre un valore positivo.

```
context Utente
inv: id > 0
```

3.2.2 Password sicura

Descrizione: La Password dell'Utente deve essere di almeno 6 caratteri.

```
context Utente
inv: password -> size() >= 6
```

3.3 Classe Utente Registrato

3.3.1 Punti sempre positivi

Descrizione: Il numero di punti di un utente registrato deve essere sempre un valore positivo.

```
context UtenteRegistrato
inv: punti >= 0
```

3.3.2 Riscattare premi

Descrizione: Per riscattare un premio il numero di punti di un utente deve avere un numero di punti maggiore o uguale a quello richiesto.

```
context UtenteRegistrato::riscattaPremio(p:Premio) :  Voucher
pre:  self.punti >= p.costoPunti
```

3.4 Classe Operatore Ecologico

3.4.1 Controllo Matricola

Descrizione: L'operazione di controllo fa un check per assicurarsi che la matricola non sia vuota.

```
context OperatoreEcologico
inv:  matricola <> ""
```

3.5 Classe Bidone

3.5.1 Livello di Riempimento

Descrizione: Il Livello di riempimento deve essere sempre compreso tra 0 e 100.

```
context Bidone
inv:  livelloRiempimento >= 0 and livelloRiempimento <= 100
```

3.6 Classe Sensore

3.6.1 Livello Batteria

Descrizione: Il Livello di carica della Batteria deve essere sempre compreso tra 0 e 100.

```
context Sensore
inv:  livelloBatteria >= 0 and livelloBatteria <= 100
```

3.6.2 Capacità Residua

Descrizione: La Capacità residua di contenimento non può essere mai negativa.

```
context Sensore
inv:  capacitàResidua >= 0
```

3.6.3 Ultimo Aggiornamento

Descrizione: L'ultimo aggiornamento deve avere una data passata o uguale a quella presente.

```
context Sensore
inv:  self.ultimoAggiornamento <= DateTime::now()
```

3.7 Classe Segnalazione

3.7.1 Descrizione non vuota

Descrizione: Il campo Descrizione della Segnalazione non può essere vuoto.

```
context Segnalazione
inv:  descrizione <> ""
```

3.7.2 Data Corretta

Descrizione: La segnalazione deve avere una data passata o uguale a quella presente.

```
context Segnalazione
inv:  dataInvio <= DateTime::now()
```

3.8 Classe Premio

3.8.1 Costo sempre positivo

Descrizione: Il costo di un Premio deve essere sempre positivo.

```
context Premio
inv:  costoPunti > 0
```

3.9 Classe Voucher

3.9.1 Data di scadenza sempre futura

Descrizione: La Data di scadenza del Voucher deve essere sempre futura rispetto al momento in cui lo si è ricevuto.

```
context Voucher
inv:  dataScadenza > Date::now()
```

3.9.2 Il codice non deve essere vuoto

Descrizione: Il codice associato al Voucher non deve essere mai vuoto.

```
context Voucher
inv:  codice <> ""
```